```
!pip install -q faiss-cpu transformers torch pandas
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 23.8/23.8 MB 47.0 MB/s eta 0:00:00
```

```
from google.colab import files
uploaded=files.upload()
```

```
Choose Files movies.csv
movies.csv(text/csv) - 494431 bytes, last modified: 4/7/2023 - 100% done
Saving movies.csv to movies (1).csv
```

```
import pandas as pd
file_name=list(uploaded.keys())[0]
df=pd.read_csv("/content/movies (1).csv")
print("Dataset loaded successfully")
df.head()
```

```
Dataset loaded successfully
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

Next steps: Generate code with df | New interactive sheet

```
TEXT_COLUMN = df.columns[0]
item_texts=df[TEXT_COLUMN].astype(str).tolist()
print("\nTotal items:",len(item_texts))
print("Sample item: ",item_texts[0])
```

```
Total items: 9742
Sample item:  1
```

```
from transformers import AutoTokenizer,AutoModel
import torch
import numpy as np
import faiss
device="cuda" if torch.cuda.is_available() else "cpu"
model_name="sentence-transformers/all-MiniLM-L6-v2"
tokenizer=AutoTokenizer.from_pretrained(model_name)
encoder=AutoModel.from_pretrained(model_name).to(device)
encoder.eval()
```

```
model.safetensors: 100%                                                 90.9M/90.9M [00:02<00:00, 661MB/s]
Loading weights: 100%                                                   103/103 [00:00<00:00, 437.27it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
BertModel(
  (embeddings): BertEmbeddings(
    (word_embeddings): Embedding(30522, 384, padding_idx=0)
    (position_embeddings): Embedding(512, 384)
    (token_type_embeddings): Embedding(2, 384)
    (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (encoder): BertEncoder(
    (layer): ModuleList(
      (0-5): 6 x BertLayer(
        (attention): BertAttention(
          (self): BertSelfAttention(
            (query): Linear(in_features=384, out_features=384, bias=True)
            (key): Linear(in_features=384, out_features=384, bias=True)
            (value): Linear(in_features=384, out_features=384, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (output): BertSelfOutput(
            (dense): Linear(in_features=384, out_features=384, bias=True)
            (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
        (intermediate): BertIntermediate(
          (dense): Linear(in_features=384, out_features=1536, bias=True)
          (intermediate_act_fn): GELUActivation()
        )
        (output): BertOutput(
          (dense): Linear(in_features=1536, out_features=384, bias=True)
          (LayerNorm): LayerNorm((384,), eps=1e-12, elementwise_affine=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
      )
    )
  )
  (pooler): BertPooler(
    (dense): Linear(in_features=384, out_features=384, bias=True)
    (activation): Tanh()
  )
)
```

```
def encode_texts(texts,batch_size=16):
  all_embeddings=[]
  with torch.no_grad():
    for i in range(0,len(texts),batch_size):
      batch=texts[i:i+batch_size]
      tokens=tokenizer(
          batch,
          padding=True,
          truncation=True,
          return_tensors="pt"
      ).to(device)
      outputs=encoder(**tokens)
      embeddings=outputs.last_hidden_state.mean(dim=1)
      embeddings=embeddings.cpu().numpy().astype("float32")
      all_embeddings.append(embeddings)
    return np.vstack(all_embeddings)
item_embeddings=encode_texts(item_texts)
faiss.normalize_L2(item_embeddings)
print("\nEmbeddings shape:",item_embeddings.shape)
```

```
Embeddings shape: (9742, 384)
```

```
embedding_dim=item_embeddings.shape[1]
index=faiss.IndexFlatIP(embedding_dim)
index.add(item_embeddings)
print("FAISS index built with", index.ntotal,"items")
```

```
FAISS index built with 9742 items
```

```
def get_similarity_items(item_id,k=5):
  query_vector=item_embeddings[item_id:item_id+1]
  scores,indices = index.search(query_vector,k)
  return scores[0],indices[0]
```

```
query_id=0
scores,idxs=get_similarity_items(query_id,k=5)
print("\nQUERY ITEM:")
print(item_texts[query_id])
print("\nSMILAR ITEMS:")
for score,idx in zip(scores,idxs):
  print(f"-> {item_texts[idx]} (score+{score:.3f})")
```

```
QUERY ITEM:
1

SMILAR ITEMS:
-> 1 (score+1.000)
-> 2 (score+0.805)
-> 3 (score+0.791)
-> 4 (score+0.769)
```